

Métodos de Otimização em Contexto Industrial

Otimização não linear com restrições

MATLAB - Comando `fmincon`

Ana Maria A. C. Rocha

Departamento de Produção e Sistemas

Universidade do Minho

arocha@dps.uminho.pt

Ano letivo 2023/24

Comando fmincon

- calcula o mínimo de problemas de otimização diferenciáveis
- mínimo de uma função multivariável sujeita a restrições não lineares
- variáveis de números reais
- usa um dos quatro métodos: 'interior-point', 'sqp', 'active-set', 'trust-region-reflective'

Para ver as opções disponíveis no algoritmo por defeito ('interior point'):

```
optimoptions('fmincon')
```

Para ver a estrutura das opções de otimização disponíveis:

```
optimset('fmincon')
```

Para ver as opções de um algoritmo específico, p.ex. 'SQP':

```
optimoptions('fmincon','Algorithm','sqp')
```

MATLAB - Comando fmincon

Comando fmincon

- O `fmincon` implementa um método baseado em gradientes que foi concebido para trabalhar em problemas cujas funções objetivo e de restrição são ambas contínuas e têm primeiras derivadas contínuas.
- Notação Matlab

minimizar $fun(x)$

s.a $c(x) \leq 0$

$ceq(x) = 0$

$Ax \leq B$

$Aeqx = Beq$

$LB \leq x \leq UB$

restrições **não lineares** de desigualdade

restrições **não lineares** de igualdade

restrições **lineares** de desigualdade

restrições **lineares** de igualdade

restrições de limites simples

Sintaxe de utilização

```
[x,fval,exitflag,output] = fmincon(fun,x0,A,B,Aeq,Beq,LB,UB,rest,opt)
```

Sintaxe de utilização

```
[x,fval,exitflag,output] = fmincon(fun,x0,A,B,Aeq,Beq,LB,UB,rest,opt)
```

Argumentos de entrada:

fun - é a função objetivo a minimizar (definir como uma função Matlab).

x0 - é a aproximação inicial.

A - matriz dos coeficientes das restrições lineares de desigualdade.

B - vetor dos termos independentes das restrições lineares de desigualdade.

Aeq - matriz dos coeficientes das restrições lineares de igualdade.

Beq - vetor dos termos independentes das restrições lineares de igualdade.

LB - vetor dos limites simples inferiores.

UB - vetor dos limites simples superiores.

rest - é a função das restrições não lineares de igualdade e desigualdade.

opt - opções de controlo do processo de otimização.

Sintaxe de utilização: opt

Algorithm	A algoritmo de otimização entre: 'interior-point' (default) 'trust-region-reflective' (requer que seja dado o gradiente) 'sqp', 'sqp-legacy' ou 'active-set'
DerivativeCheck	Compara Jacobiano dado com a aproximação por diferenças finitas
Diagnostics	Diagnóstico da função a ser otimizada ('off' default)
DiffMaxChange	Diferença máxima nas variáveis para as diferenças finitas (default=0.1)
DiffMinChange	Diferença mínima nas variáveis para as diferenças finitas (default=1e-8)
Display	Nível de apresentação off - não apresenta nada iter - apresenta resultados em cada iteração notify - apresenta resultado se a função não convergir final - (default) apresenta apenas o resultado final
FunValCheck	Testa se a função objetivo não é número complexo, Inf ou NaN on - apresenta erro off - (default) não apresenta erro

Sintaxe de utilização: opt

GradObj	Gradiente da função objetivo dado pelo utilizador ('off' default)
MaxFunEvals	Nº máximo de cálculos da função - (default = 100*NumVar)
MaxIter	Nº máximo de iterações - (default = 1000)
OutputFcn	Especifica uma ou mais funções que o processo de otimização pode invocar, em cada iteração.
PlotFcns	Representa várias medidas do progresso do algoritmo, ao longo das iterações. @optimplotx - representa o ponto atual. @optimplotfunccount - nº de cálculos da função objetivo. @optimplotfval - valor da função objetivo. @optimplotconstrviolation - violação máxima das restrições. @optimplotstepsize - comprimento do passo. @optimplotfirstorderopt - medida otimalidade de 1ª ordem.
TolFun	Tolerância de paragem relativamente à função objetivo (default = 1e-6)
TolCon	Tolerância de paragem relativamente à violação das restrições (default = 1e-6)
TolX	Tolerância de paragem relativamente a x - (default = 1e-6)

Sintaxe de utilização

```
[x,fval,exitflag,output] = fmincon(fun,x0,A,B,Aeq,Beq,LB,UB,rest,opt)
```

Argumentos de saída:

x - é a solução do problema.

fval - é o valor da função objetivo em x.

exitflag - descreve valores de saída do fminunc.

output - estrutura com informação acerca do processo de otimização

exitflag

1	condições de otimalidade de 1ª ordem satisfeitas. Converge.
2	um alteração em x é menor que a tolerância TolX.
3	uma alteração na função objetivo é menor que a tolerância TolFun.
0	o número de iterações atingiu o MaxIter ou o MaxFunEvals.
-2	indica que a função não convergiu para uma solução admissível.
> 0	indica que convergiu para uma solução admissível.
0	o número de iterações excedeu o MaxIter ou o MaxFunEvals.
< 0	indica que não convergiu para uma solução admissível.

Sintaxe de utilização

```
[x,fval,exitflag,output] = fmincon(fun,x0,A,B,Aeq,Beq,LB,UB,rest,opt)
```

output

output - estrutura com informação acerca do processo de otimização

iterations	número de iterações realizadas
funcCount	número de cálculos da função
constrviolation	máximo da violação das restrições (algoritmos "active-set" e "interior-point")
stepsize	indica o comprimento do passo final (algoritmos "active-set" e "interior-point")
algorithm	algoritmo usado
firstorderopt	valor da medida de otimalidade de 1ª ordem
cgiterations	número total de iterações PCG
message	apresenta a mensagem de saída

Algoritmos de otimização

Otimização através do `fmincon`

- Interior-Point - o método lida com problemas de grandes dimensões e esparsos, bem como com problemas pequenos e densos.
- SQP and SQP-Legacy
- Active-Set
- Trust-Region-Reflective - requer que seja dado o gradiente

Síntese

- Sem mudar qualquer opção $\Leftrightarrow$ **Método interior-point**
- Usando `GradObj = 'on'` $\Leftrightarrow$ **Método 'trust-region-reflective'**
- Os algoritmos 'sqp' e 'sqp-legacy' do `fmincon` são semelhantes ao algoritmo 'active-set'. Ou seja, o `fmincon` utiliza o método de programação quadrática sequencial (SQP), e em cada iteração, resolve um subproblema de programação quadrática (QP). A estimativa da Hessiana da função Lagrangiana em cada iteração é atualizada com base na fórmula BFGS.

Exercício 1

Considere o seguinte problema de otimização com restrições

$$\begin{array}{ll}\min & (x_1^2 + 3x_2)^2 \\ \text{s.a} & x_1 - x_2^3 \geq 2 \\ & x_1 + x_2 = 5 \\ & x_1 \geq 0\end{array}$$

Use como aproximação inicial o vetor $(1, 2)$.

Resolva o problema, usando o comando mais adequado do Matlab, e:

- ① sem alterar qualquer opção.
- ② visualizando os resultados em cada iteração.
- ③ usando como tolerância de paragem 3 iterações.
- ④ considerando $\text{tolfun} = 1 \times 10^{-10}$.
- ⑤ considerando $\text{tolx} = 1 \times 10^{-10}$.
- ⑥ considerando $\text{tolfun} = 1 \times 10^{-8}$ e $\text{tolx} = 1 \times 10^{-8}$.
- ⑦ visualizando o evoluir da função objetivo ao longo das iterações.

Exercício 2

Implemente em MATLAB o seguinte problema

$$\begin{aligned} \min \quad & (x_1 + 3x_2 + x_3)^2 + 4(x_1 - x_2)^2 \\ \text{s.a} \quad & 6x_2 + 4x_3 - x_1^3 \geq 3 \\ & x_1 - 3x_2 - 4x_3 \geq 4 \\ & x_1 + x_2 + x_3 = 1 \end{aligned}$$

Resolva-o, usando como aproximação inicial o vetor $(0.1, 0.7, 0.2)$, e:

- ❶ sem alterar qualquer opção.
- ❷ visualizando os resultados em cada iteração e considerando $\text{tolfun} = 1 \times 10^{-2}$ e $\text{tolx} = 1 \times 10^{-2}$.
- ❸ visualizando graficamente o valor do minimizante e o número de cálculos da função objetivo, ao longo das iterações.
- ❹ visualizando graficamente o evoluir da função objetivo e da violação das restrições, ao longo das iterações.
 - ❶ O processo convergiu?
 - ❷ Qual foi o número de iterações realizadas?
 - ❸ Qual foi o valor do minimizante encontrado? E o valor do mínimo?

Exercício 3

A empresa “Paralelo&Wood SA” pretende confeccionar uma caixa em madeira de base quadrada, aberta no topo, como mostra a figura.



Pretende-se que o volume da caixa seja de 3 m^3 . Determine as dimensões ótimas da caixa que minimizam a quantidade de madeira a usar.

Considere a seguinte aproximação inicial $(x_1, x_2)^{(1)} = (1, 1)$.

Exercício 4

Numa determinada localidade rural, o Sr. Joaquim possui uma vaca que pasta numa determinada área delimitada por uma corda presa a uma estaca fixa no solo e que restringe o pasto do animal a uma área circular.

No entanto, próximo dessa área passa um pequeno riacho que o Sr. Joaquim quer aproveitar para que a vaca beba lá água.

Sabendo que a estaca se encontra num ponto cujas coordenadas são $(3,3)$ e que o riacho pode ser definido pela reta $x + 2y = 4$, determine o comprimento mínimo que a corda deve ter de modo a que a vaca consiga chegar ao curso de água.

Resolução do Exercício 1

Considere o seguinte problema de otimização com restrições

$$\begin{array}{ll}\min & (x_1^2 + 3x_2)^2 \\ \text{s.a} & x_1 - x_2^3 \geq 2 \\ & x_1 + x_2 = 5 \\ & x_1 \geq 0\end{array}$$

Resolva-o através do comando `fmincon` do Matlab e usando como aproximação inicial o vetor $(1, 2)$.

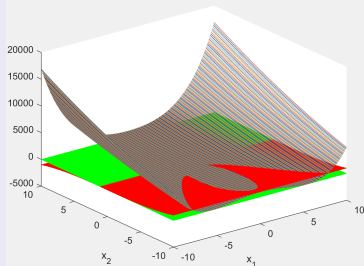
O problema deve ser reescrito para atender à notação do Matlab.

Assim, têm-se:

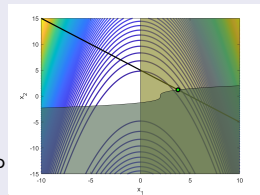
$$\begin{array}{ll}\min & (x_1^2 + 3x_2)^2 \\ \text{s.a} & -x_1 + x_2^3 + 2 \leq 0 \quad \Leftrightarrow \quad \text{restrição não linear de desigualdade} \\ & x_1 + x_2 = 5 \quad \Leftrightarrow \quad \text{restrição linear de igualdade} \\ & x_1 \geq 0 \quad \Leftrightarrow \quad \text{limites nas variáveis}\end{array}$$

Resolução do Exercício 1 (cont.)

Representação gráfica



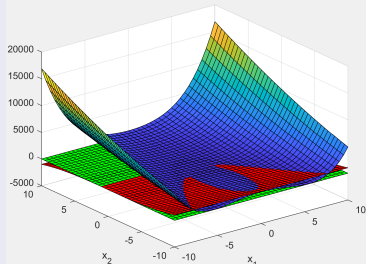
```
[x1,x2]=meshgrid(-10:0.05:10,-10:0.05:10);  
fx=(x1.^2+3*x2).^2;  
plot3(x1,x2,fx);  
hold on;  
c1=x1-x2.^3-2;  
plot3(x1,x2,c1,'r');  
c2=x1+x2-5;  
plot3(x1,x2,c2,'g');  
xlabel('x_1');  
ylabel('x_2');
```



```
[x1,x2]=meshgrid(-10:0.05:10,-15:0.05:15);  
fx=(x1.^2+3*x2).^2;  
contour(x1,x2,fx,100,'linewidth',2); %100 linhas contorno  
hold on  
plot((-10:0.05:10),5-(-10:0.05:10),'k','linewidth',2); % c2 -> x2=5-x1  
fill([10 0 0 10],[15 15 -15 -15],[0.5 0.5 0.1],'facealpha',0.6) % x1 >= 0  
x = -10:0.05:10; y = -15:0.05:15;  
xc2=y.^3+2; % x >= y.^3+2  
fill([xc2 10 10],[y y(end) y(1)],[0.3 0.4 0.3],'facealpha',0.6);  
xlim([-10,10])  
xlabel('x_1'); ylabel('x_2');  
plot(3.7866,1.2134,'ok','linewidth',2,'MarkerEdgeColor','k','MarkerFaceColor','g')
```

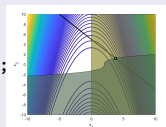
Resolução do Exercício 1 (cont.)

Outra forma de representação gráfica



```
[x1,x2]=meshgrid(-10:0.5:10,-10:0.5:10);  
fx=(x1.^2+3*x2).^2;  
c1=x1-x2.^3-2;  
c2=x1+x2-5;  
surf(x1,x2,fx)  
hold on  
surf(x1,x2,c1,'FaceColor','r')  
surf(x1,x2,c2,'FaceColor','g')  
xlabel('x_1');  
ylabel('x_2');
```

```
fx=@(x1,x2)(x1.^2+3*x2).^2;  
fcontour(fx,[-10 10 -10 10],'LineWidth',2,'LevelStep',100);  
hold on  
fimplicit(@(x1,x2) x1+x2-5,[-10 10 -10 10],'k-','LineWidth',2); % c2 -> x2=5-x1  
[x1,x2]=meshgrid(-10:0.05:10,-10:0.05:10);  
fill([10 0 0 10],[10 10 -10 -10],[0.5 0.5 0.1],'facealpha',0.6) % x1 >= 0  
x = -10:0.05:10; y = -10:0.05:10;  
xc2=y.^3+2; % x >= y.^3+2  
fill([xc2 10 10],[y y(end) y(1)],[0.3 0.4 0.3],'facealpha',0.6);  
xlim([-10,10])  
xlabel('x_1'); ylabel('x_2');  
plot(3.7866,1.2134,'ok','linewidth',2,'MarkerEdgeColor','k','MarkerFaceColor','g')
```



Resolução do Exercício 1 (cont.)

Resolva o problema, usando o comando mais adequado do Matlab, e:

- 1 sem alterar qualquer opção.

```
x0=[1;2]; %Ponto inicial
%Definicao da restricao linear de igualdade
Aeq=[1 1];
Beq=[5];
%Definicao das restricoes lineares de desigualdade
A=[];
B=[];
%Definicao das restricoes de limites simples
LB=[0;-Inf];
UB=[];
%Invocacao do comando de otimizacao
[x,fval,exitflag,output]=fmincon(@PNL,x0,A,B,Aeq,Beq,LB,UB,@rest_PNL)
%funcao objetivo
function [f] = PNL(x)
    f=(x(1)^2+3*x(2))^2;
end
%funcao das restricoes não lineares
function [c,ceq] = rest_PNL(x)
    c=-x(1)+x(2)^3+2;
    ceq=[];
end
```

Resolução do Exercício 1 (cont.)

- 1 visualizando os resultados em cada iteração.

```
x0=[1;2]; %Ponto inicial
%Definicao da restricao linear de igualdade
Aeq=[1 1];
Beq=[5];
%Definicao das restricoes lineares de desigualdade
A=[];
B=[];
%Definicao das restricoes de limites simples
LB=[0;-Inf];
UB=[];
%opcoes
opt=optimset('display','iter');
%Invocacao do comando de otimizacao
[x,fval,exitflag,output]=fmincon(@PNL,x0,A,B,Aeq,Beq,LB,UB,@rest_PNL,opt)
%funcao objetivo
function [f] = PNL(x)
    f=(x(1)^2+3*x(2))^2;
end
%funcao das restricoes não lineares
function [c,ceq] = rest_PNL(x)
    c=-x(1)+x(2)^3+2;
    ceq=[];
end
```

Resolução do Exercício 1 (cont.)

- ❶ considerando $\text{tolfun} = 1 \times 10^{-10}$.

```
...  
%alterar opcoes  
opt=optimset('tolfun',1e-10);  
...
```

- ❷ considerando $\text{tolx} = 1 \times 10^{-10}$.

```
%alterar opcoes  
opt=optimset('tolx',1e-10);
```

- ❸ considerando $\text{tolfun} = 1 \times 10^{-8}$ e $\text{tolx} = 1 \times 10^{-8}$.

```
%alterar opcoes  
opt=optimset('tolx',1e-8,'tolfun',1e-8);
```

- ❹ utilizando o algoritmo SQP.

```
%alterar opcoes  
opt=optimset('Algorithm','sqp');
```

- ❺ visualizando o evoluir da função objetivo ao longo das iterações.

```
%alterar opcoes  
opt=optimset('plotfcns',@optimplotfval);
```

Resolução do Exercício 2

Implemente em MATLAB o seguinte problema

$$\begin{array}{ll}\min & (x_1 + 3x_2 + x_3)^2 + 4(x_1 - x_2)^2 \\ \text{s.a} & 6x_2 + 4x_3 - x_1^3 \geq 3 \\ & x_1 - 3x_2 - 4x_3 \geq 4 \\ & x_1 + x_2 + x_3 = 1\end{array}$$

Resolva o problema usando como aproximação inicial $(0.1, 0.7, 0.2)$.

O problema deve ser reescrito para atender à notação do Matlab.

Assim, têm-se que:

$$\begin{array}{ll}\min & (x_1 + 3x_2 + x_3)^2 + 4(x_1 - x_2)^2 \Leftrightarrow \text{restrição não linear desigualdade} \\ \text{s.a} & -6x_2 - 4x_3 + x_1^3 + 3 \leq 0 \Leftrightarrow \text{restrição não linear de desigualdade} \\ & -x_1 + 3x_2 + 4x_3 \leq -4 \Leftrightarrow \text{restrição linear de desigualdade} \\ & x_1 + x_2 + x_3 = 1 \Leftrightarrow \text{restrição linear de igualdade}\end{array}$$

Resolução do Exercício 2 (cont.)

- 1 sem alterar qualquer opção.

```
x0=[0.1;0.7;0.2];
%Definicao da restricao linear de igualdade
Aeq=[1 1 1];
Beq=[1];
%Definicao das restricoes lineares de desigualdade
A=[-1 3 4];
B=[-4];
%Definicao das restricoes de limites simples
LB=[];
UB=[];
%Invocacao do comando de otimizacao
[x,fval,exitflag,output]=fmincon(@prob,x0,A,B,Aeq,Beq,LB,UB,@rest_prob)
%funcao objetivo
function [f] = prob(x)
    f=(x(1)+3*x(2)+x(3))^2+4*(x(1)-x(2))^2;
end
%funcao das restricoes
function [c, ceq] = rest_prob(x)
    c = [-6*x(2)-4*x(3)+ x(1)^3+3];
    ceq = [];
end
```

Resolução do Exercício 2 (cont.)

- ① resultados em cada iteração e $\text{tolfun} = 1 \times 10^{-2}$ e $\text{tolx} = 1 \times 10^{-2}$.

```
x0=[0.1;0.7;0.2];
%Definicao da restricao linear de igualdade
Aeq=[1 1 1];
Beq=[1];
%Definicao das restrições lineares de desigualdade
A=[-1 3 4];
B=[-4];
%Definicao das restrições de limites simples
LB=[];
UB=[];
%opcoes
opt=optimset('display','iter','tolx',1e-2,'tolx',1e-2);
%Invocação do comando de otimização
[x,fval,exitflag,output]=fmincon(@prob,x0,A,B,Aeq,Beq,LB,UB,@rest_prob,opt)
%funcao objetivo
function [f] = prob(x)
    f=(x(1)+3*x(2)+x(3))^2+4*(x(1)-x(2))^2;
end
%funcao das restricoes
function [c, ceq] = rest_prob(x)
    c = [-6*x(2)-4*x(3)+ x(1)^3+3];
    ceq = [];
end
```

Resolução do Exercício 2 (cont.)

- 1 visualizando graficamente o valor do minimizante e o número de cálculos da função objetivo, ao longo das iterações.

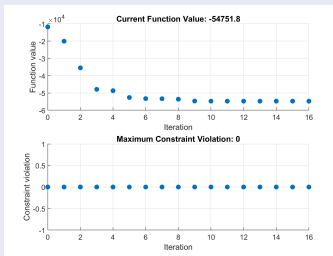
```
%alterar opcoes
```

```
opt=optimset('plotfcn',{@optimplotx @optimplotfunccount});
```

- 2 visualizando graficamente o evoluir da função objetivo e da violação das restrições, ao longo das iterações.

```
%alterar opcoes
```

```
opt=optimset('plotfcn',{@optimplotfval @optimplotconstrviolation});
```



Resolução do Exercício 3

O volume do paralelepípedo é dado por $v = x_1^2 x_2$ em que x_1 é o comprimento dos lados e x_2 é a altura. A função objetivo é minimizar a quantidade de madeira a usar para fazer a caixa, por isso é minimizar a área da superfície da caixa.

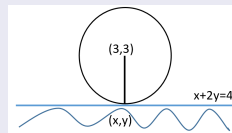
$$\begin{aligned} \min_{x \in \mathbb{R}^2} f(x) &\equiv x_1^2 + 4x_1x_2 \\ \text{s.a} \quad &x_1^2x_2 = 3 \\ &x_1, x_2 \geq 0 \end{aligned}$$

```
A=[];      B=[];
Aeq=[];    Beq=[];
LB=[0;0]; UB=[];
x0=[1; 1];
[x,fval,exitflag,output] = fmincon(@fun,x0,A,B,Aeq,Beq,LB,UB,@rest)
function [f] = fun(x)
    f=x(1)^2+4*x(1)*x(2);
end
function [c,ceq] = rest(x)
    c=[];
    ceq=x(1)^2*x(2)-3;
end
```


Resolução do Exercício 4

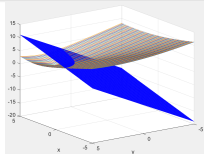
$$\min_{x \in \mathbb{R}^2} f(x) \equiv \sqrt{(x-3)^2 + (y-3)^2}$$

s.a $x + 2y = 4$

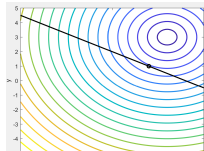


```
A=[];      B=[];
Aeq=[1 2]; Beq=[4];
LB=[0;0]; UB=[];
x0=[1; 1];

[x,fval,exitflag,output] = fmincon(@fun,x0,A,B,Aeq,Beq,LB,UB,[])
function [f] = fun(x)
    f=sqrt((x(1)-3)^2+(x(2)-3)^2);
end
```



```
[x,y]=meshgrid(-5:0.1:5,-5:0.1:5);
fx=sqrt((x-3).^2+(y-3).^2);
c1=x+2*y-4;
plot3(x,y,fx,x,y,c1,'b');
xlabel('x'); ylabel('y'); grid
%%
```



```
[x,y]=meshgrid(-5:0.1:5,-5:0.1:5);
fx=sqrt((x-3).^2+(y-3).^2);
contour(x,y,fx,20,'linewidth',2); %faz 20 linhas de contorno
hold on
fimplicit(@(x,y) x+2*y - 4,[-5 5 -5 5],'k-','LineWidth',2)
xlabel('x'); ylabel('y');
```